



CS6P05NM Project

Interim Report

Capacity-Aware Sprint Planning System with Predictive Overload Detection

Name: Vithanage Angelo Prabajith Perera

ID Number: 25024547

Date: Friday, 9th January 2026

First Supervisor: Prof. Ruvan Abeysekara

Second Supervisor:

Declaration

Module: CS6P05NM Deadline: Saturday, 9th January 2026 at 03:00PM

Module Leader: Prof. Ruvan Abeysekara

Student ID: 25024547

PLAGIARISM

You are reminded that there exist regulations concerning plagiarism. Extracts from these regulations are printed below. Please sign below to say that you have read and understand these extracts:

(Signature:)



Date: 09/01/25

This header sheet should be attached to the work you submit. No work will be accepted without it.

Extracts from University *Regulations* on Cheating, Plagiarism and Collusion

Section 2.3: "The following broad types of offence can be identified and are provided as indicative examples..."

- (i) Cheating: including taking unauthorised material into an examination; consulting unauthorised material outside the examination hall during the examination; obtaining an unseen examination paper in advance of the examination; copying from another examinee; using an unauthorised calculator during the examination or storing unauthorised material in the memory of a programmable calculator which is taken into the examination; copying coursework.
- (ii) Falsifying data in experimental results.
- (iii) Personation, where a substitute takes an examination or test on behalf of the candidate. Both candidate and substitute may be guilty of an offence under these Regulations.
- (iv) Bribery or attempted bribery of a person thought to have some influence on the candidate's assessment.
- (v) Collusion to present joint work as the work solely of one individual.
- (vi) Plagiarism, where the work or ideas of another are presented as the candidate's own.
- (vii) Other conduct calculated to secure an advantage on assessment.
- (viii) Assisting in any of the above.

Some notes on what this means for students:

1. Copying another student's work is an offence, whether from a copy on paper or from a computer file, and in whatever form the intellectual property being copied takes, including text, mathematical notation and computer programs.
2. Taking extracts from published sources *without attribution* is an offence. To quote ideas, sometimes using extracts, is generally to be encouraged. Quoting ideas is achieved by stating an author's argument and attributing it, perhaps by quoting, immediately in the text, his or her name and year of publication, e.g. " $E = mc^2$ (Einstein 1905)". A *references* section at the end of your work should then list all such references in alphabetical order of authors' surnames. (There are variations on this referencing system which your tutors may prefer you to use.) If you wish to quote a paragraph or so from published work then indent the quotation on both left and right margins, using an italic font where practicable, and introduce the quotation with an attribut

Abstract

Software development teams operating in agile environments often face challenges related to workload imbalance, over-allocation of developers, and reduced sprint predictability, particularly when managing multiple projects and unplanned tasks concurrently. Traditional sprint planning tools primarily focus on task tracking and workflow management, offering limited support for real-time capacity analysis and early detection of overload risks. This can lead to unrealistic sprint commitments and frequent disruptions during sprint execution.

This project focuses on the development of a Capacity-Aware Sprint Planning System with Predictive Overload Detection, designed to support more informed sprint planning through explicit modelling of developer capacity. The proposed system aims to enhance decision-making by analysing planned and unplanned work against available capacity and identifying potential overload conditions before they negatively impact sprint outcomes. The solution is implemented as a web-based application using modern full-stack technologies, with an emphasis on analytical workload modelling rather than basic task management.

This interim report presents the background and motivation for the project, outlines the work completed to date, and details the remaining tasks required to achieve the project objectives. The initial phases of problem analysis, requirement definition, and system design have been completed in alignment with the project plan, and technical implementation has commenced as scheduled. The report also provides a review of current progress, confirming that the project remains on track for completion within the allocated timeframe.

Contents

Declaration.....	1
Abstract.....	2
Contents.....	3
1. Introduction.....	4
2. Background.....	6
3. Work Completed.....	10
4. Further Work.....	12
5. Progress Review.....	14
6. References.....	16
7. Bibliography.....	17

1. Introduction

Modern software development teams frequently operate in fast-paced environments where multiple projects, clients, and deadlines compete for limited developer capacity. In such settings, sprint planning often focuses on task allocation without sufficient consideration of individual workload limits, existing commitments, or the impact of unplanned work. As a result, teams commonly experience developer over-allocation, frequent sprint disruptions, and reduced predictability in delivery outcomes. These challenges are further amplified in multi-project environments, where developers contribute to several streams of work simultaneously.

This project addresses these issues through the development of a Capacity-Aware Sprint Planning System with Predictive Overload Detection. The system aims to support sprint planning by explicitly modelling developer capacity and identifying potential overload risks before and during sprint execution. Rather than functioning as a general task management tool, the proposed system focuses on providing decision support by analysing planned and unplanned work against available capacity, enabling more informed sprint commitments and workload allocation.

The project builds upon the original proposal for a sprint and capacity management system and refines it by introducing predictive and analytical elements that enhance its academic and technical depth as recommended. The solution is designed as a web-based application using modern full-stack technologies, with an emphasis on capacity modelling, workload visibility, and early detection of sprint risk conditions.

This interim report presents the progress made toward achieving the project objectives and outlines the work completed to date. The report is structured as follows: the Background section summarises relevant work in agile sprint planning and capacity management; the Work Completed section details the analysis, design decisions, and initial development undertaken so far; the Further Work section identifies the remaining tasks required to complete the project; and the Progress Review provides an evaluation of current progress against the project plan, including any risks and mitigation strategies.

2. Background

Agile methodologies have become the dominant approach for managing modern software development projects, particularly in environments characterised by changing requirements and frequent delivery cycles. Central to Agile frameworks such as Scrum is sprint planning, where teams commit to a set of tasks to be completed within a fixed time period. Effective sprint planning requires not only accurate task estimation, but also a realistic understanding of developer capacity and workload constraints. When capacity is not adequately considered, teams are prone to over-commitment, reduced delivery quality, and frequent sprint spillovers (Cohn, 2005).

In practice, many software development teams rely on established project management tools such as Jira, ClickUp, and Azure DevOps to manage tasks and sprints. These platforms provide comprehensive features for backlog management, issue tracking, and reporting. However, their primary focus is often on task status and workflow progression rather than on dynamic capacity analysis. While these tools allow teams to assign story points or estimated hours, they typically assume stable availability and do not actively model the impact of competing projects, context switching, or unplanned work introduced during a sprint. As a result, capacity-related risks are often identified only after sprint execution has begun, rather than during planning.

Research in agile project management highlights the importance of aligning sprint commitments with actual team capacity. Studies examining sprint planning consistency have shown that teams with more structured and data-informed planning practices tend to demonstrate higher delivery stability and improved sprint outcomes

(Zander and Meboldt, 2024). Similarly, empirical analyses of agile development practices indicate that workload balance and planning discipline play a significant role in determining overall project success (Ghimire and Charters, 2022). These findings suggest that tools supporting sprint planning should move beyond basic task tracking and incorporate analytical mechanisms that assist teams in evaluating the feasibility of sprint commitments.

Another challenge commonly identified in agile environments is the handling of unplanned or ad-hoc tasks. Such tasks frequently arise due to production issues, urgent client requests, or operational dependencies, and often disrupt existing sprint plans. Traditional sprint management approaches provide limited support for assessing the impact of these interruptions on delivery timelines. Recent research has explored adaptive sprint planning approaches that incorporate risk awareness and dynamic adjustment of sprint scope, demonstrating potential benefits in maintaining predictability under uncertain conditions (Marchwicka and Marchwicki, 2023; Nazir et al., 2022).

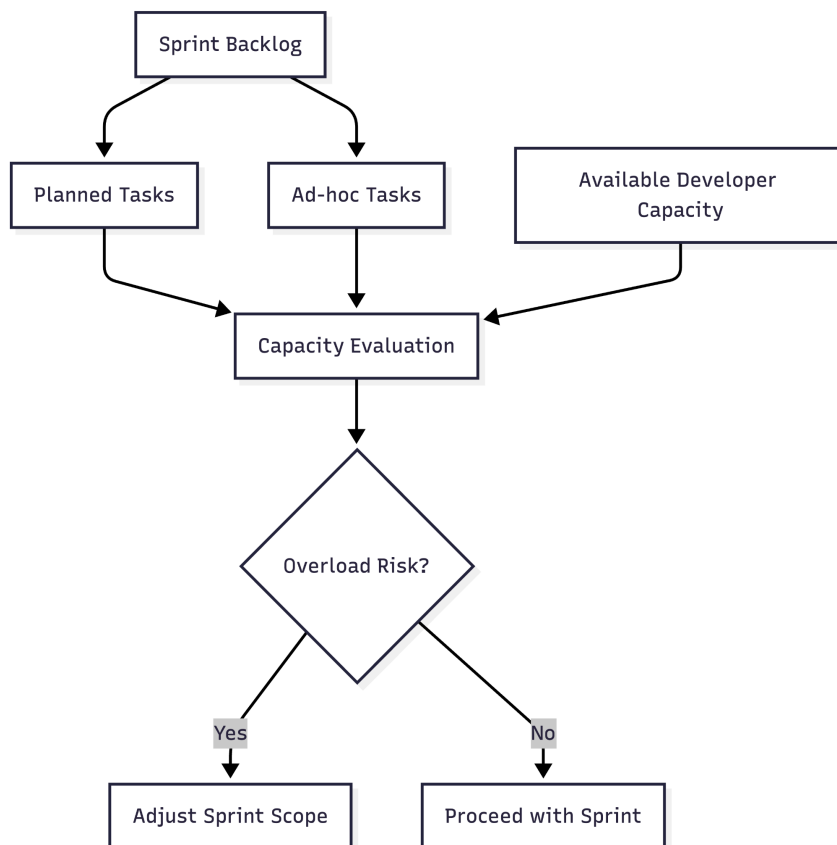


Figure 1: Conceptual overview of sprint capacity evaluation and overload detection

Despite the availability of mature project management platforms and ongoing research into agile planning practices, there remains a gap between theoretical capacity-aware planning models and their practical implementation within everyday development workflows. Existing tools tend to prioritise flexibility and configurability, sometimes at the expense of clear, real-time workload visibility and decision support. This gap highlights the need for systems that explicitly model developer capacity, account for planned and unplanned work, and provide early indicators of overload risk during sprint planning and execution.

This project is situated within this context and seeks to address the identified limitations by proposing a capacity-aware sprint planning system that integrates predictive overload detection. By combining established agile principles with

analytical workload modelling, the system aims to support more informed decision-making and improve sprint reliability in multi-project development environments.

3. Work Completed

The project has progressed in accordance with the work breakdown structure and Gantt chart defined during the proposal stage. The initial phases focused on problem analysis, requirement definition, and system design, establishing a solid foundation before moving into technical implementation. At the time of this interim report, the first three planned stages are nearing completion, and development activities for the core technical components have commenced.

The review and problem analysis phase involved a detailed examination of challenges associated with sprint planning and developer workload management in multi-project agile environments. This stage refined the project focus from general task tracking toward capacity-aware sprint planning and predictive overload detection. Particular attention was given to identifying limitations in existing tools and practices, especially in relation to workload visibility and the handling of unplanned work during active sprints.

Following this, requirement gathering and finalisation was conducted to clearly define the functional and non-functional requirements of the system. Core functional requirements were identified, including sprint creation, task allocation, developer capacity modelling, and overload risk identification. Non-functional considerations such as system usability, scalability, and data consistency were also outlined to ensure the proposed solution remains practical and robust. These requirements were reviewed and refined to maintain alignment with the project aims and agreed scope.

The high-level system design phase focused on defining the overall

architecture of the proposed solution. Key design decisions included the adoption of a web-based, full-stack architecture and the separation of concerns between the frontend, backend services, and database layer. Initial data models were designed to represent developers, tasks, sprints, and capacity metrics, supporting future implementation of workload calculations and predictive logic. This stage also involved outlining the core workflows for sprint planning and capacity evaluation.

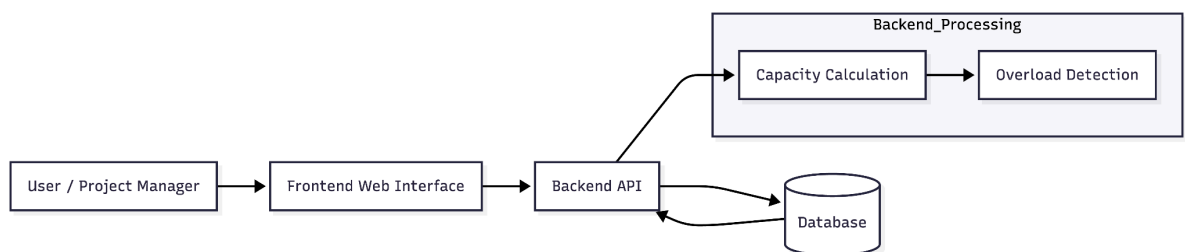


Figure 2: High-level system architecture of the capacity-aware sprint planning system

With these foundational stages substantially completed, work has now progressed into the technical implementation phase. Initial development efforts have begun on backend architecture and database design, providing the groundwork for implementing task management and capacity-related functionality. This transition marks the shift from conceptual design to system construction, in line with the original project schedule.

Overall, the work completed to date demonstrates a structured and methodical approach to addressing the project problem, ensuring that analytical and design considerations are firmly established before advancing further into development and evaluation stages.

4. Further Work

The remaining work for this project follows the later stages outlined in the original project plan and Gantt chart and focuses primarily on system implementation, integration, evaluation, and final documentation. These activities build directly upon the completed analysis and design phases and are structured to ensure steady progress toward a fully functional and evaluated system.

The next phase of work will concentrate on backend development, including the implementation of core application programming interfaces for sprint management, task allocation, user authentication, and capacity data handling. This stage will also involve completing the database schema and implementing the logic required to calculate developer capacity and identify potential overload conditions based on sprint commitments and unplanned work.

Following backend development, attention will shift to frontend implementation, with the creation of user interfaces for sprint planning, task management, and capacity visualisation. This includes developing views that clearly present workload distribution and highlight overload risks in an intuitive manner. The frontend and backend components will then be integrated to ensure seamless data flow and consistent system behaviour.

Once the primary functionality is in place, integration and internal testing will be conducted to verify correctness, stability, and data integrity across the system. This will be followed by an evaluation and refinement phase, during which usability

feedback will be gathered and scenario-based testing will be performed to assess how effectively the system supports capacity-aware sprint planning and responds to ad-hoc task insertion.

The final stages of the project will involve documentation and submission preparation, including the completion of the final report, system documentation, and any supporting materials required for assessment. These activities are scheduled toward the end of the project timeline and allow sufficient time for review and refinement prior to submission.

Based on the progress achieved to date and adherence to the planned schedule, the remaining work is considered achievable within the allocated timeframe. No major revisions to the project plan are currently required - however, the plan will continue to be reviewed as implementation progresses to ensure risks are identified early and addressed appropriately.

5. Progress Review

At the time of writing this interim report, the project is progressing in line with the schedule defined in the original project plan. The initial phases, including problem analysis, requirement finalisation, and high-level system design, have been completed within the planned timeframes. The transition into technical implementation has also begun as scheduled, indicating that the overall project timeline remains achievable.

A review of the Gantt chart confirms that the work completed to date aligns with the planned milestones. The early focus on analysis and design has reduced the risk of major scope changes during implementation, providing a stable foundation for the remaining development tasks. This structured approach has helped ensure that progress is measurable and that dependencies between project phases are clearly managed.

The remaining work primarily consists of backend and frontend development, followed by integration, testing, and evaluation. While these stages represent the most technically intensive portion of the project, they have been allocated sufficient time in the project plan. No significant delays or unforeseen obstacles have been encountered so far, and no revisions to the project schedule are currently required.

Potential risks have been identified, particularly in relation to the complexity of capacity modelling and the integration of predictive overload detection logic. These risks are being managed by implementing core functionality incrementally and validating assumptions early through testing and refinement. This approach reduces the likelihood of late-stage issues impacting the overall delivery timeline.

Based on the progress achieved and the current rate of development, the project is considered to be on target for completion within the allocated timeframe. The project plan will continue to be reviewed regularly to ensure that any emerging risks are identified promptly and addressed in a timely manner.

6. References

- Cohn, M. (2005) *Agile Estimating and Planning*. Upper Saddle River, NJ: Pearson Education.
- Ghimire, D. and Charters, S. (2022) ‘The impact of Agile development practices on project outcomes’, *Software*, 1(3), pp. 188–205. Available at: <https://www.mdpi.com/2674-113X/1/3/12> (Accessed: 5 January 2026).
- Marchwicka, E. and Marchwicki, T. (2023) *Adaptive Sprint Planning Based on Risk Management*. SSRN Working Paper. Available at: https://papers.ssrn.com/sol3/papers.cfm?abstract_id=4571373 (Accessed: 5 January 2026).
- Nazir, S., Price, B., Surendra, N.C. and Kopp, K. (2022) ‘Adapting agile development practices for hyper-agile environments’, *Information Technology and Management*, 23, pp. 315–330. Available at: <https://link.springer.com/article/10.1007/s10799-022-00370-y> (Accessed: 4 January 2026).
- Zander, M.O. and Meboldt, M. (2024) ‘Navigating the unknown: Introduction of an analysis metric for the sprint plan and sprint review in educational agile development projects’, *Proceedings of the 52nd SEFI Conference*. Available at: <https://www.research-collection.ethz.ch/entities/publication/659bafea-62f0-4a65-afdf-bd623ed0da27> (Accessed: 3 January 2026).

7. Bibliography

Atlassian (2023) *Sprint Planning: What it is and How to Do it Right*. Available at: <https://www.atlassian.com/blog/agile/sprint-planning-atlassian> (Accessed: 1 December 2025).

Scrum.org (2023) *Sprint Capacity Planning for Scrum Teams: A Practical Guide*. Available at: <https://www.scrum.org/resources/blog/sprint-capacity-planning-scrum-teams-practical-guide> (Accessed: 10 November 2025).

Mountain Goat Software (2022) *Should a Team Assign Work During Sprint Planning?*. Available at: <https://www.mountaingoatsoftware.com/blog/should-a-team-assign-work-during-sprint-planning> (Accessed: 1 December 2025).

Beck, K. (2000) *Extreme Programming Explained: Embrace Change*. Boston, MA: Addison-Wesley.

Rubin, K.S. (2012) *Essential Scrum: A Practical Guide to the Most Popular Agile Process*. Upper Saddle River, NJ: Addison-Wesley.